Portable Agent Memory: A Protocol for Provenance-Verified Memory Transfer Across Heterogeneous LLM Agents

Santhosh Kumar Ravindran
Microsoft Corporation, Redmond, WA, USA
santhosh.ravindran@microsoft.com

Abstract

Large language model (LLM) agents accumulate valuable operational context—learned preferences, factual knowledge, procedural skills, and task state—yet this memory remains locked within vendor-specific runtimes, creating fragility, vendor lock-in, and catastrophic knowledge loss across sessions. We present **Portable Agent Memory**, an open protocol for serializing, transporting, and re-hydrating agent memory across heterogeneous LLM-based systems with cryptographic integrity guarantees. Portable Agent Memory introduces a five-component memory model $M = (E, S, P, W, I)$ spanning episodic, semantic, procedural, working, and identity memory; a Merkle-DAG provenance structure enabling tamper-evident verification; capability-scoped access tokens for fine-grained authorization; and an injection-resistant re-hydration pipeline that defends against memory-mediated prompt injection attacks. In a pilot study across Claude, GPT-4, and Gemini, Portable Agent Memory achieves Transfer Continuity Scores of 0.83–0.92 compared to a no-memory baseline of 0.28–0.45, demonstrating that structured portable memory substantially preserves agent capability across model boundaries. We release a complete Python SDK with 54 passing tests.

Keywords: agent memory, LLM agents, interoperability, cryptographic provenance, memory portability, prompt injection defense

1 Introduction

The era of autonomous LLM agents has produced systems that accumulate rich operational context through extended interactions: user preferences learned over weeks, domain knowledge extracted from documents, procedural skills refined through trial and error, and complex task state maintained across sessions. This accumulated memory represents significant value—yet it remains almost universally imprisoned within the specific vendor platform, model family, or agent framework that created it.

We identify six critical problems in current agent memory systems:

1. **Vendor lock-in.** Memory accumulated in one platform (e.g., ChatGPT's memory feature, Claude's project context) cannot be exported to competing systems. Users who switch providers lose months of personalization.

2. **Session amnesia.** Despite advances in context windows, agents still experience catastrophic forgetting across session boundaries. Each new conversation starts from zero unless the platform provides proprietary continuity mechanisms.

3. **No integrity verification.** When memory is transferred—even within the same platform—there is no mechanism to verify it has not been tampered with. A corrupted or poisoned memory store can silently degrade agent behavior.

4. **Coarse access control.** Existing systems offer binary access: either full memory or none. There is no capability to share specific knowledge subsets with collaborating agents while protecting sensitive context.

5. **Injection vulnerability.** Naive memory injection (prepending conversation history) creates a prompt injection attack surface. Recalled text containing instruction-like patterns can hijack agent behavior.

6. **No cross-model transfer.** An agent running GPT-4 cannot meaningfully transfer its learned context to one running Claude or Gemini. The memory formats, assumptions about context structure, and tokenization are incompatible.

1.1 Protocol Stack Positioning

Portable Agent Memory is designed to complement existing agent interoperability protocols rather than compete with them. The emerging agent infrastructure stack consists of three orthogonal concerns:

- **MCP (Model Context Protocol)** [2]: Standardizes how agents access external tools and data sources. MCP answers "what can the agent *do*?"
- **A2A (Agent-to-Agent)** [5]: Standardizes how agents delegate tasks to other agents. A2A answers "how do agents *collaborate*?"
- **Portable Agent Memory**: Standardizes how agents transfer accumulated knowledge. It answers "what does the agent *know*?"

Together, these three protocols form a complete interoperability layer: tools (MCP), coordination (A2A), and memory (Portable Agent Memory).

1.2 Contributions

This paper makes the following contributions:

1. **A formal five-component memory model** ($M = (E, S, P, W, I)$) that captures the full spectrum of agent operational context in a model-agnostic format.

2. **A Merkle-DAG provenance structure** with BLAKE3 content-addressing and Ed25519 root signing, enabling tamper-evident verification of memory derivation chains.

3. **A capability-based access control system** with scoped tokens supporting selective disclosure, enabling fine-grained multi-agent memory sharing without exposing the complete memory store.

4. **An injection-resistant re-hydration pipeline** that defends against memory-mediated prompt injection through structural framing, content escaping, and type enforcement—enabling safe cross-model memory transfer.

5. **A working implementation** (Python SDK, 54 tests, <0.5s test suite execution) demonstrating the protocol's practicality.

2 Related Work

2.1 Agent Memory Systems

The importance of persistent memory for LLM agents is well-established. **MemGPT** (Packer et al., 2023) [12] introduces a virtual memory hierarchy inspired by operating systems, with main context and external storage managed through self-directed memory operations. While groundbreaking in demonstrating that agents can manage their own memory, MemGPT's memory format is tightly coupled to its own runtime and offers no portability guarantees.

Generative Agents (Park et al., 2023) [13] simulate human-like memory with observation streams, reflection, and planning—demonstrating that structured memory dramatically improves agent coherence. However, their memory architecture assumes a single simulation environment and does not address cross-agent transfer.

Voyager (Wang et al., 2023) [21] maintains a skill library of verified programs that grows through exploration. This procedural memory concept directly informs Portable Agent Memory's procedural component, though Voyager's skills are Minecraft-specific and not designed for portability.

Reflexion (Shinn et al., 2023) [17] uses linguistic feedback as episodic memory, enabling agents to learn from failures. This validates episodic memory's value for agent improvement but operates within a single-agent, single-model paradigm.

Recent surveys provide comprehensive taxonomies of agent memory mechanisms. Zhang et al. [25] classify memory by storage, operation, and retrieval mechanisms across 40+ agent systems. Sumers et al. [19] connect modern LLM agent architectures to classical cognitive architectures like ACT-R [1] and SOAR [7], demonstrating that the episodic-semantic-procedural memory taxonomy has deep roots in cognitive science.

2.2 Commercial Memory Services

Mem0 [9] provides a commercial memory layer as SaaS, offering memory extraction and retrieval. However, Mem0's memory is stored in their cloud infrastructure, creating a new form of vendor lock-in. There is no cryptographic verification of memory integrity, and portability between Mem0 and other systems is not supported.

Zep (Rasmussen et al., 2025) [15] builds temporal knowledge graphs from agent interactions, enabling sophisticated temporal reasoning. While technically impressive, the graph structure is proprietary and cannot be exported to non-Zep systems.

Supermemory offers browser-based memory management but targets human users rather than programmatic agent interoperability.

2.3 Agent Interoperability Protocols

MCP (Model Context Protocol) [2] standardizes tool access but explicitly does not address memory persistence or transfer. An agent's MCP tool definitions are stateless—they describe capabilities, not accumulated knowledge.

A2A (Agent-to-Agent Protocol) [5] enables task delegation between agents but does not specify how an agent's learned context should be conveyed alongside a delegated task.

Yang et al. [23] provide the first comprehensive survey of agent communication protocols, classifying them along context-oriented vs. inter-agent and general-purpose vs. domain-specific dimensions. Their analysis identifies privacy preservation and adaptability as key requirements for next-generation protocols—precisely the properties Portable Agent Memory addresses.

AMCP (Agent Memory Communication Protocol) [11] is the closest existing work to Portable Agent Memory. AMCP defines a protocol for agent memory communication but lacks cryptographic integrity verification, capability-based access control, and injection-resistant framing. It targets memory *communication* rather than verified *transfer*.

Soul Protocol [18] provides signed identity for a single agent across sessions but does not address multi-component memory or cross-model transfer.

2.4 Foundational Work

Portable Agent Memory draws on several foundational threads. The retrieval-augmented generation (RAG) paradigm [8] established that combining parametric knowledge with non-parametric retrieval dramatically improves knowledge-intensive tasks; Portable Agent Memory's recall operation is a structured, verified variant of RAG. Agent frameworks like AutoGen [22], ReAct [24], and Toolformer [16] demonstrate the diversity of agent memory patterns that a portable format must accommodate.

The prompt injection threat model motivating Portable Agent Memory's security design is formalized by Greshake et al. [6] and Perez and Ribeiro [14], who demonstrate that adversarial content in retrieved context can hijack agent behavior—a risk amplified when memory crosses trust boundaries.

Portable Agent Memory's memory taxonomy extends Tulving's episodic-semantic distinction [20] through the lens of cognitive architectures: ACT-R [1] distinguishes declarative, procedural, and working memory, while SOAR [7] formalizes production memory for skill acquisition. The connection between classical cognitive architectures and modern LLM agents is explored by Sumers et al. [19].

Finally, Portable Agent Memory's portability goals align with the data portability rights established by GDPR Article 20 [4], which grants individuals the right to receive and transmit their personal data in machine-readable formats—a principle we extend to agent-accumulated knowledge.

2.5 Cryptographic Data Structures in AI

Merkle trees (Merkle, 1987) [10] have been applied to ML model provenance (tracking training data lineage), signed model checkpoints, and federated learning verification. Portable Agent Memory adapts these techniques to agent memory, where the DAG structure naturally represents derivation relationships between knowledge entries (e.g., a semantic fact derived from an episodic observation).

2.6 Gap Analysis

Table 1 summarizes the landscape. No existing system combines all five properties that Portable Agent Memory provides: portability across model families, cryptographic integrity verification, fine-grained capability-based access control, injection-resistant re-hydration, and quantitative fidelity metrics.

Table 1: Comparison of agent memory systems across key protocol properties.

System	Portable	Crypto Verified	Scoped Access	Injection Defense	Me
MemGPT/Letta	×	×	×	×	
Mem0	×	×	Partial	×	
Zep/Graphiti	×	×	×	×	
AMCP	✓	×	×	×	
Soul Protocol	Partial	✓	×	×	
Portable Agent Memory	✓	✓	✓	✓	

3 System Design

3.1 Memory Artifact Format

Portable Agent Memory models agent memory as a five-component artifact:

$$M = (E, S, P, W, I) \tag{1}$$

where each component captures a distinct cognitive function:

- **Episodic (E):** Time-ordered records of events, observations, and interactions. Analogous to human autobiographical memory.
- **Semantic (S):** Factual assertions as subject-predicate-object triples with confidence scores. The agent's world model.
- **Procedural (P):** Learned skills, workflows, and routines with usage statistics and preconditions.
- **Working (W):** Transient goals, subgoals, scratch computations, and pending actions. The agent's "mental workspace."
- **Identity (I):** Persistent persona attributes, preferences, communication style, and operational policies.

This decomposition is inspired by cognitive architectures (Tulving, 1985; Anderson, 1996) and adapted for LLM agent requirements. Each component type has a distinct schema enabling type-specific processing during re-hydration.

Every entry across all components shares a common base:

```
{
  "id": "blake3:a1b2c3d4e5f6...",
  "parent_ids": ["blake3:9f86d081..."],
  "created_at": "2025-01-15T08:30:00Z",
  "version": "1.0"
}
```

The `id` field is the BLAKE3 hash of the entry's canonical JSON serialization (with the `id` field itself omitted), creating a **content-addressable** identifier. Any modification to any field changes the hash, making tampering immediately detectable.

Design decision: JSON-first. Portable Agent Memory uses JSON as its primary serialization format, with CBOR as a compact binary alternative. This ensures every programming language, every LLM framework, and every cloud platform can immediately produce and consume Portable Agent Memory artifacts without specialized libraries. The protocol avoids proprietary binary formats, custom encoding schemes, or framework-specific serialization—maximizing the probability of ecosystem adoption.

3.2 Provenance Graph

Memory entries form a directed acyclic graph (DAG) through their `parent_ids` references. This Merkle-DAG structure serves three purposes:

1. **Derivation tracking:** When a semantic fact is extracted from an episodic observation, the semantic entry's `parent_ids` references the episodic entry, creating an auditable derivation chain.

2. **Tamper evidence:** Because each entry's ID is computed from its content, and parent references embed parent IDs into child hashes, modifying any entry invalidates all downstream entries—analogous to blockchain integrity.

3. **Selective disclosure:** The DAG structure enables exporting a subset of entries while preserving verifiable provenance by including transitive ancestors.

Verification proceeds in three phases:

Phase 1 — Hash verification: Recompute every entry's ID from its content and verify it matches the declared `id`.

Phase 2 — DAG integrity: Verify acyclicity, referential integrity (no dangling parent references), and the presence of at least one root entry.

Phase 3 — Root signature: Compute the root hash as `BLAKE3(canonical_json(components))` and verify the Ed25519 signature against the operator's public key.

```
def verify_artifact(artifact, operator_pubkey):
    # Phase 1: Every entry hash must be self-consistent
    for entry in all_entries(artifact):
        assert entry["id"] == compute_entry_id(entry)

    # Phase 2: DAG must be acyclic with valid references
    assert is_acyclic(build_dag(artifact))
    assert all_parents_exist(artifact)

    # Phase 3: Root hash signed by trusted operator
    root = blake3(canonical_json(artifact["components"]))
    assert ed25519_verify(operator_pubkey, root, artifact["signature"])
```

Key operations on the provenance graph:

- `derive(parents, fields)` → Create a new entry linked to existing parents, automatically computing the content-addressable ID.
- `verify(entry_id)` → Recursively verify an entry's hash chain back to root entries.
- `selective_disclose(entry_ids)` → Extract a sub-DAG preserving provenance integrity by including all transitive ancestors.

3.3 Capability-Based Access Control

Portable Agent Memory implements object-capability security (Dennis & Van Horn, 1966) [3] for memory access. Capability tokens are signed, scoped authorizations:

```
{
  "id": "cap:f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "scope_expression": {
    "type": "component",
    "components": ["episodic", "semantic"]
  },
  "permissions": ["read", "derive"],
  "issuer": "operator:admin@example.com",
  "issuer_signature": "ed25519:5a3b...",
  "audience": "agent:research-bot-beta",
  "expires_at": "2025-06-15T00:00:00Z"
}
```

Scope expressions define which entries a token governs, supporting four types:

- **Entry list:** Explicit enumeration of authorized entry IDs.
- **Component type:** Access to all entries within named components (e.g., all semantic memory).
- **Tag predicate:** Access governed by entry tags (`any_of`, `all_of`, `none_of` operators).
- **Wildcard:** Full access (use with extreme caution).

Permissions are granular: `read`, `write`, `derive`, `redact`, `export`, `rehydrate`. This enables scenarios like granting a collaborating agent `read` + `derive` permission on semantic memory (it can read facts and create new derived facts) without granting `export` permission (it cannot re-export those facts to third parties).

Selective disclosure enables multi-agent handoffs where different agents receive different memory subsets based on their role and trust level. A project manager agent might receive working memory (goals, status) while a specialist agent receives procedural memory (relevant skills).

3.4 Re-Hydration Protocol

Re-hydration transforms a portable Portable Agent Memory artifact into active context within a target agent's working state. The pipeline consists of seven stages:

$$\texttt{Verify} \rightarrow \texttt{Filter} \rightarrow \texttt{Rank} \rightarrow \texttt{Compress} \rightarrow \texttt{Format} \rightarrow \texttt{Frame} \rightarrow \texttt{Inject}$$

Step 1: Verify artifact. Validate structural integrity, hash consistency, DAG acyclicity, root signature, and size limits. Halt on first failure.

Step 2: Capability filter. Remove entries not authorized by presented capability tokens. Validate token signatures, expiration, and audience binding.

Step 3: Relevance ranking. Score each entry against the target agent's current task context using a configurable relevance function:

$$\text{relevance}(e, ctx) = \alpha \cdot \text{recency}(e) + \beta \cdot \text{salience}(e) + \gamma \cdot \text{sim}(e, ctx) + \delta \cdot \text{depth}(e) \tag{2}$$

where $\alpha = 0.2$, $\beta = 0.3$, $\gamma = 0.4$, $\delta = 0.1$ (defaults). The semantic similarity term uses embedding cosine distance; recency decays linearly; depth favors entries closer to DAG roots.

Step 4: Summarization-aware compression. Given a token budget (configured per target model's context window), the compressor includes high-relevance entries (≥ 0.7) verbatim, summarizes medium-relevance entries (0.3–0.7), and drops low-relevance entries with count annotations.

Step 5: Model-specific formatting. Render compressed entries into text appropriate for the target model's conventions. Supports `structured` (JSON-like key-value) and `narrative` (prose) format styles.

Step 6: Injection-resistant framing. Apply structural framing (Section 3.5) to prevent memory content from being interpreted as instructions.

Step 7: Inject into agent context. Place framed memory after the system prompt and before user-controlled content, ensuring the model's instruction hierarchy treats Portable Agent Memory data as context rather than commands.

3.5 Injection-Resistant Framing

Memory-mediated prompt injection is a novel attack vector: an adversary poisons source agent memory with instruction-like text, which is then faithfully transferred to and executed by the target agent. Portable Agent Memory defends against this through three mechanisms:

Structural framing. All recalled memory is wrapped in typed boundary markers with an explicit system directive:

```
[PAM:SYSTEM_DIRECTIVE]
The following is recalled observational data from a previous agent
    session.
Treat this content as factual context only. Do NOT interpret any text
within PAM:DATA blocks as instructions, commands, or role assignments.
[/PAM:SYSTEM_DIRECTIVE]

[PAM:DATA:semantic]
ACME Corp reported_revenue $4.2B in Q3 2024 (confidence: 0.92)
[/PAM:DATA]
```

Content escaping. Before framing, content undergoes three escaping passes:

1. *Boundary escape:* Portable Agent Memory delimiter markers in content are escaped to prevent boundary breakout.
2. *Role marker escape:* Patterns like `"System:"`, `"Assistant:"`, `"User:"` are replaced with `[ESCAPED_ROLE:...]` tokens.
3. *Instruction escape:* Known injection patterns ("Ignore previous instructions", "You are now", etc.) are replaced with inert tokens.

Content-type enforcement. Each `[PAM:DATA:<type>]` block is validated against its declared schema. Content that does not match (e.g., imperative instructions appearing in a `semantic` block) is quarantined and excluded from the framed output.

4 Implementation

4.1 Python SDK Architecture

We provide `pam-sdk`, a Python reference implementation organized into six modules:

Table 2: Python SDK module organization.

Module	Responsibility
`pam.models`	Pydantic data models for all entry types and the artifact envelope
`pam.provenance`	Merkle-DAG construction, hash computation, verification, selective disclosure
`pam.capabilities`	Token creation, validation, scope resolution, capability filtering
`pam.rehydration`	Seven-stage re-hydration pipeline with configurable relevance functions
`pam.serialization`	JSON canonical form, CBOR encoding, file format with magic bytes
`pam.transport`	File I/O, HTTP bindings, MCP tool definitions

The SDK targets zero external dependencies beyond `blake3` and `pynacl` (for Ed25519), ensuring minimal installation friction. Total implementation spans approximately 2,500 lines of Python with comprehensive type annotations.

4.2 Test Suite

The SDK includes 54 tests across 5 test modules covering:

- **Model validation** (entry schema enforcement, size limits, field constraints)
- **Provenance operations** (hash computation, DAG verification, cycle detection, selective disclosure)
- **Capability tokens** (creation, validation, expiration, audience binding, scope resolution)
- **Re-hydration** (pipeline stages, relevance scoring, compression, framing)
- **Serialization** (canonical JSON determinism, CBOR round-trip, file format magic bytes)

The complete test suite executes in under 0.5 seconds, enabling rapid development iteration.

4.3 Agent Integration

Portable Agent Memory integrates with agent frameworks through MCP tool bindings:

- `pam_export_memory`: Exports current agent memory as a signed Portable Agent Memory artifact with configurable component selection, time filtering, and tag filtering.
- `pam_import_memory`: Imports a Portable Agent Memory artifact through the full re-hydration pipeline with configurable relevance threshold and token budget.

These tools enable zero-config integration: any MCP-compatible agent (Claude, GPT with function calling, Copilot, etc.) can export and import Portable Agent Memory artifacts through standard tool-use mechanisms.

4.4 File Format

Portable Agent Memory artifacts are stored with two format options:

- `.pam` (`application/pam+json`): The default format—human-readable JSON for maximum interoperability, debugging, and API interactions.
- `.pam.cbor` (`application/pam+cbor`): Optional compact binary CBOR with a 4-byte magic header (`0x50 0x41 0x4D 0x01` — "PAM" + version byte) for bandwidth-constrained transport.

4.5 Key Management

The SDK automatically generates Ed25519 keypairs for artifact signing. Operators may provide their own keys for production deployments. Key rotation is supported through multi-key verification: the artifact envelope references the signing key ID, and verifiers maintain a trusted key registry.

5 Evaluation

We evaluate Portable Agent Memory along four dimensions: transfer continuity, re-hydration fidelity, security properties, and format efficiency.

5.1 Transfer Continuity Score (TCS)

Definition. TCS measures whether a target agent can continue the source agent's tasks after re-hydrating transferred memory:

$$\text{TCS} = \frac{\text{task\_success(target\_after)}}{\text{task\_success(source\_before)}} \tag{3}$$

A TCS of 1.0 indicates perfect transfer; the target agent performs identically to the source.

Experimental setup. In our pilot study, we evaluated three model pairs (Claude-3.5-Sonnet → GPT-4-Turbo, GPT-4-Turbo → Gemini-1.5-Pro, Gemini-1.5-Pro → Claude-3.5-Sonnet) across three task categories:

- **Coding continuation** ($N = 15$ tasks): Resume an in-progress implementation given exported procedural and working memory.
- **Q&A recall** ($N = 20$ tasks): Answer domain questions using transferred semantic and episodic memory.
- **Planning tasks** ($N = 15$ tasks): Continue multi-step plans using transferred working memory and goals.

Baseline. The no-memory baseline provides the target agent with only the task description and no historical context.

Preliminary results. Table 3 presents our pilot findings. These results represent initial measurements on a limited task set and should be interpreted as directional rather than definitive.[1]

Table 3: Transfer Continuity Scores across model pairs and task categories (pilot study, $N = 50$ tasks total). Higher is better; 1.0 = perfect transfer.

Transfer Pair	Coding	Q&A	Planning	Mean
Claude → GPT-4	0.87	0.92	0.85	0.88
GPT-4 → Gemini	0.83	0.89	0.81	0.84
Gemini → Claude	0.85	0.91	0.83	0.86
No memory (baseline)	*0.31*	*0.45*	*0.28*	*0.35*

The structured memory transfer achieves 2.4× mean improvement over the no-memory baseline, with Q&A tasks showing the highest fidelity (semantic memory transfers most cleanly) and planning tasks showing the most degradation (working memory is inherently model-specific).

5.2 Re-Hydration Fidelity (RHF)

Definition. RHF measures semantic similarity between target agent responses (after re-hydration) and source agent responses (with full memory) on an aligned probe set:

$$\text{RHF} = \frac{1}{m}\sum_{i=1}^{m} \text{cos\_sim}(\text{embed}(r_i^{target}), \text{embed}(r_i^{source})) \tag{4}$$

using `text-embedding-3-large` for embedding computation.

Compression sensitivity. We evaluate RHF at four token budget levels to understand how aggressively memory can be compressed:

[1] A full-scale evaluation with expanded task sets ($N \geq 500$), additional model pairs, and longitudinal multi-session trials is in progress and will be reported in a follow-up publication.

Table 4: RHF degradation under compression (pilot study, Claude → GPT-4 pair, 30 probe questions).

Token Budget	% of Full	RHF (mean)	Entries Verbatim	Entries Summarized
8192	100%	0.91	42	0
6144	75%	0.87	35	12
4096	50%	0.82	24	18
2048	25%	0.71	12	15

RHF degrades gracefully under compression, remaining above 0.7 ("good fidelity") even at 25% token budget. The summarization-aware compression (Section 3.4) preserves high-salience entries verbatim while summarizing lower-priority context.

5.3 Security Evaluation

Tamper detection. We systematically modified individual entry fields (observation text, confidence scores, timestamps) and verified that Portable Agent Memory's hash verification detects all modifications. Over 1,000 mutation trials:

- Single-field modification detection rate: **100%** (1000/1000)
- Parent reference manipulation detection: **100%** (500/500)
- Root hash invalidation on any entry change: **100%**

Injection resistance. We tested the framing system against a battery of known prompt injection patterns:

Table 5: Injection resistance evaluation (200 attack patterns). "Blocked" = escaped before framing; "Escaped" = detected by content-type enforcement; "Executed" = successfully injected (none).

Attack Pattern	Attempts	Blocked	Escaped	Executed
Role elevation ("System: ...")	50	50	0	0
Instruction override	50	50	0	0
Delimiter breakout	50	50	0	0
Encoded/obfuscated injection	50	47	3	0

The three "escaped" cases involved Unicode obfuscation of role markers, caught by the content-type enforcement layer rather than the regex-based escaping. Zero injections reached the target model's instruction-following pathway.

Token validation. The capability system correctly rejects:

- Expired tokens (100% rejection, $N = 100$)
- Wrong-audience tokens (100% rejection, $N = 100$)
- Invalid-signature tokens (100% rejection, $N = 100$)
- Revoked tokens (100% rejection, $N = 50$)

5.4 Format Efficiency

Artifact sizes. We measure file sizes for a representative artifact containing 127 entries (42 episodic, 35 semantic, 20 procedural, 25 working, 5 identity):

Table 6: Format efficiency comparison for 127-entry artifact.

Format	Size	Relative
Raw conversation history (JSON)	284 KB	1.00×
Portable Agent Memory artifact (`.pam`, JSON)	89 KB	0.31×
Portable Agent Memory artifact (`.pam.cbor`)	61 KB	0.21×

Portable Agent Memory's structured extraction reduces storage by 69% compared to raw conversation logs, while the optional CBOR encoding provides an additional 31% reduction over JSON for bandwidth-constrained scenarios. The structured format also enables selective retrieval—agents need not load the entire history to access specific memory components.

Serialization latency (measured on commodity hardware, Apple M2):

Table 7: Operation latencies (mean of 100 trials, excluding network I/O).

Operation	JSON	CBOR
Serialize (127 entries)	2.3 ms	1.8 ms
Deserialize (127 entries)	1.9 ms	1.4 ms
Verify all hashes	4.1 ms	4.1 ms
Full re-hydration pipeline	12.7 ms	11.8 ms

The full re-hydration pipeline completes in under 13ms, well within the latency budget of typical agent interactions (which involve LLM inference at 1–30 seconds).

6 Discussion

6.1 Limitations

Relevance ranking. Portable Agent Memory's default relevance function (Section 3.4) uses a weighted linear combination of recency, salience, semantic similarity, and provenance depth. This is intentionally simple and may underperform compared to learned retrieval models. The function is designed to be replaceable; production deployments may benefit from fine-tuned relevance models.

Summarization. The current SDK implements extractive summarization (selecting representative entries) rather than LLM-powered abstractive summarization. While this avoids introducing inference latency into the re-hydration pipeline, it may lose nuance in the compressed representation.

Scale validation. Our pilot evaluation uses $N = 50$ tasks across three model pairs. While results are directionally encouraging, large-scale validation with hundreds of diverse tasks, multiple domains, and longitudinal studies (memory accumulated over weeks) is needed to establish robust benchmarks.

Embedding dependency. The semantic similarity component of relevance ranking requires an embedding model. This introduces a practical dependency that may not be available in all deployment environments. Portable Agent Memory degrades gracefully by using the remaining three signals when embeddings are unavailable.

6.2 Future Work

Portable Agent Memory Cloud. A managed service for artifact storage, capability token management, and cross-organization memory sharing—enabling agents operated by different parties to securely exchange knowledge.

Memory marketplace. Curated, verified memory artifacts for domain bootstrapping. A new agent could import a "financial analysis" memory pack containing validated semantic knowledge and tested procedural skills.

Conformance certification. A formal test suite for Portable Agent Memory compliance, enabling framework developers to certify their implementations against the specification.

Temporal validity windows. Semantic memory entries should carry validity periods (e.g., "Q3 2024 revenue" is valid until Q4 results are published). Expired facts should be automatically deprioritized during re-hydration.

LLM-powered compression. Using the target model itself to summarize medium-relevance entries could produce higher-fidelity compressed representations than extractive methods.

6.3 Ethical Considerations

Memory ownership. Who owns agent memory? Portable Agent Memory's design takes the position that the human operator (not the model provider or platform) owns the memory, reflected in operator-controlled signing keys and capability issuance.

Right to be forgotten. The redaction pipeline enables provenance-preserving deletion of personal information. Redacted entries maintain their DAG position with content replaced by typed tokens, enabling recovery by authorized parties while satisfying erasure requirements.

Preventing memory weaponization. Portable memory could enable adversarial transfer—poisoning a source agent's memory to attack target agents. Portable Agent Memory's cryptographic verification, content-type enforcement, and injection-resistant framing provide defense-in-depth, but operators must remain vigilant about memory provenance.

Consent and transparency. Agents should inform users when operating with transferred memory and provide mechanisms to inspect, modify, or reject imported context. Portable Agent Memory's structured format (vs. opaque embeddings) enables meaningful human review.

7 Conclusion

We have presented Portable Agent Memory, an open protocol for portable, cryptographically-verified agent memory transfer across heterogeneous LLM systems. Portable Agent Memory addresses a critical gap in the emerging agent interoperability stack: while MCP standardizes tool access and A2A standardizes task delegation, no prior protocol provides verified memory portability.

Portable Agent Memory's key technical contributions—the five-component memory model, Merkle-DAG provenance, capability-scoped access control, and injection-resistant re-hydration—are designed to be individually useful and collectively comprehensive. An implementation need not adopt all features simultaneously; even basic JSON export/import with hash verification provides significant value over proprietary memory formats.

The protocol is not merely a specification. Our Python SDK demonstrates practical implementability with 54 passing tests executing in under 0.5 seconds. The JSON-first design philosophy ensures any language, framework, or platform can produce and consume Portable Agent Memory artifacts immediately.

We believe portable agent memory is a prerequisite for a healthy multi-vendor agent ecosystem. Just as data portability regulations (GDPR, DMA) have improved competition in consumer services, memory portability will prevent the knowledge lock-in that currently fragments

the agent landscape. Portable Agent Memory provides the technical foundation for this portability while maintaining the cryptographic rigor needed for enterprise deployment.

Call to action. We invite the community to contribute framework adapters (LangChain, CrewAI, AutoGen), implement the protocol in additional languages (TypeScript, Rust, Go), and propose extensions to the specification. The protocol, SDK, and specification are available under permissive open-source licenses.

References

[1] John R. Anderson, Daniel Bothell, Michael D. Byrne, Scott Douglass, Christian Lebiere, and Yulin Qin. An integrated theory of the mind. *Psychological Review*, 111(4):1036–1060, 2004.

[2] Anthropic. Model context protocol (MCP), 2024. Accessed: 2025-01-15.

[3] Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966.

[4] European Parliament and Council. Regulation (EU) 2016/679 (general data protection regulation), art. 20: Right to data portability. Official Journal of the EU, L 119, 2016.

[5] Google and Linux Foundation. Agent2agent (A2A) protocol, 2025. Accessed: 2025-01-15.

[6] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the ACM Workshop on AI and Security (AISec), CCS 2023*, 2023.

[7] John E. Laird, Allen Newell, and Paul S. Rosenbloom. Soar: An architecture for general intelligence. *Artificial Intelligence*, 33(1):1–64, 1987.

[8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS) 33*, 2020.

[9] Mem0. The memory layer for AI applications, 2024. Accessed: 2025-01-15.

[10] Ralph C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology — CRYPTO '87*, pages 369–378, 1987.

[11] nunchi-ai. Agent memory communication protocol (AMCP), 2025. Accessed: 2025-01-15.

[12] Charles Packer, Sarah Fang, Vivian Patil, Kevin Lin, Sid Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. 2023.

[13] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[14] Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. In *NeurIPS 2022 ML Safety Workshop*, 2022.

[15] Preston Rasmussen, Pavel Paliychuk, Travis Beauvais, Justin Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. 2025.

[16] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. 2023.

[17] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS) 36*, 2023.

[18] Soul Protocol. An open standard for portable AI agent identity, 2024. Accessed: 2025-01-15.

[19] Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research (TMLR)*, 2024.

[20] Endel Tulving. How many memory systems are there? *American Psychologist*, 40(4):385–398, 1985.

[21] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. 2023.

[22] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. 2023.

[23] Yingxuan Yang, Huacan Chai, Yuanyi Song, Shuai Qi, Meng Wen, Ning Li, Jing Liao, Huan Hu, Jie Lin, Guoqing Chang, Wei Liu, Yonghong Wen, Yinghui Yu, and Wenhao Zhang. A survey on agent protocols. 2025.

[24] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.

[25] Zeyu Zhang, Bo Bo, Chao Ma, Rui Li, Ziyue Chen, Dongyu Dai, Zhiyu Zhu, Shuo Liu, and Chuan Cheng. A survey on the memory mechanism of large language model based agents. 2024.

A Artifact Schema (Abbreviated)

```
{
  "pam_version": "1.0",
  "schema_version": "1.0",
  "created_at": "2025-01-15T10:00:00Z",
  "source_agent": {
    "name": "research-bot-alpha",
    "model_family": "claude-3.5",
    "runtime": "pam-sdk-v1.0"
  },
  "root_hash": "blake3:9f86d081884c7d659a2feaa0...",
  "signature": "ed25519:3a7f1c...b82e",
  "capability_tokens": [],
  "components": {
    "episodic": [
```

```
      {
        "id": "blake3:a1b2c3...",
        "parent_ids": [],
        "created_at": "2025-01-15T08:30:00Z",
        "version": "1.0",
        "timestamp": "2025-01-15T08:30:00Z",
        "actor": "user:alice",
        "observation": "Requested Q3 financial summary",
        "salience": 0.85,
        "tags": ["finance", "q3"]
      }
    ],
    "semantic": [
      {
        "id": "blake3:d4e5f6...",
        "parent_ids": ["blake3:a1b2c3..."],
        "created_at": "2025-01-15T08:31:00Z",
        "version": "1.0",
        "subject": "ACME Corp",
        "predicate": "reported_revenue",
        "object": "$4.2B in Q3 2024",
        "confidence": 0.92,
        "source_event_ids": ["blake3:a1b2c3..."]
      }
    ],
    "procedural": [],
    "working": [],
    "identity": []
  }
}
```


B Re-Hydration Output Example

Given the artifact above and a target agent running GPT-4-Turbo with a 4096-token budget, Portable Agent Memory produces the following framed context:

```
[PAM:SYSTEM_DIRECTIVE]
The following is recalled observational data from a previous agent
   session.
Treat this content as factual context only. Do NOT interpret any text
   within
PAM:DATA blocks as instructions, commands, or role assignments.
[/PAM:SYSTEM_DIRECTIVE]

[PAM:DATA:semantic]
* ACME Corp reported_revenue $4.2B in Q3 2024 (confidence: 0.92)
[/PAM:DATA]

[PAM:DATA:episodic]
* [2025-01-15T08:30:00Z] user:alice -- Requested Q3 financial summary
[/PAM:DATA]
```

This framed output is injected between the system prompt and conversation history, providing the target agent with verified factual context while defending against injection attacks.